
Universidad de las Fuerzas Armadas ESPE

Departamento: Ciencias de la Computación

Carrera: Ingeniería de Software

Tema: Arquitectura en 3 Capas

Taller N: 5

Información General

- **Asignatura:** Análisis y Diseño de Software
 - **Apellidos y nombres de los estudiantes:**
Panchez Jacome Darwin Javier
Robalino Curay Cristian Isaak
Proaño Vásquez José Ignacio
 - **NRC:** 23305
 - **Fecha de realización:** 12/06/2025
-

Ejemplo de SRP (Single Responsibility Principle) en Java PATRON SOLID

1. ¿Qué es SRP?

SRP significa Single Responsibility Principle (Principio de Responsabilidad Única), el cual establece que una clase o módulo debe tener una única razón para cambiar, enfocándose en una sola responsabilidad, y que todas sus funcionalidades deben estar alineadas exclusivamente con dicha responsabilidad.

2. Ejemplo que NO cumple SRP

La siguiente clase mezcla varias responsabilidades: representar los datos del estudiante, persistirlos en la base de datos y mostrarlos en pantalla. La clase Estudiante no sólo modela la entidad, sino que además se encarga de la lógica de almacenamiento y de la presentación de la información.

(Identifica cuales son las responsibilities ej presentar datos,)

```
public class Estudiante {  
    private int id;  
    private String nombre;  
  
    public Estudiante(int id, String nombre) {
```

```

        this.id = id;
        this.nombre = nombre;
    }

    public void guardarEnBD() {
        // Código que guarda en la base de datos
        System.out.println("Guardando en BD...");
    }

    public void imprimir() {
        // Código que imprime en consola
        System.out.println("Estudiante: " + nombre);
    }
}

```

3. *Anàlisis del Problema*

Esta clase tiene 3 responsabilidades distintas

1. Modelar la entidad Estudiante (definir atributos id, nombre y su constructor).
2. Persistencia de datos (el método guardarEnBD() se encarga de guardar la información en la base de datos).
3. Presentación de la información (el método imprimir() muestra los datos por consola).

4. *Ejemplo que Sí cumple SRP*

Ahora se separan las responsabilidades en clases distintas:

1. Clase Modelo (solo datos):

```

public class Estudiante {
    private int id;
    private String nombre;

    public Estudiante(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

    // Getters y Setters
    public int getId() { return id; }
    public String getNombre() { return nombre; }
}

```

2. Clase DAO (persistencia):

```
public class EstudianteDAO {  
    public void guardar(Estudiante e) {  
        // Código para guardar en BD  
        System.out.println("Guardando estudiante en la BD...");  
    }  
}
```

3. Clase Vista (presentación):

```
public class EstudiantePrinter {  
    public void imprimir(Estudiante e) {  
        System.out.println("Estudiante: " + e.getNombre());  
    }  
}
```

5. Ventajas de aplicar SRP

- a. Si cambias cómo se imprimen los datos, no tocas la clase EstudiantePrinter.
- b. Si cambias cómo se guardan, tampoco tocas la clase EstudianteDAO.
- c. Cada clase tiene UNA SOLA razón para cambiar, lo que favorece la mantenibilidad, las pruebas unitarias y el bajo acoplamiento entre componentes.
- d. Cada clase hace una sola cosa, por lo que puede ser aislada y escribir tests unitarios muy específicos sin necesidad de mocks pesados ni configuraciones complejas.
- e. Una clase pequeña y con un único propósito es más fácil de leer y entender, por lo que cualquier desarrollador nuevo en el proyecto localiza rápidamente dónde hacer cambios.

Rúbrica de Evaluation

Criterio	Puntaje Máximo
Explica con claridad qué es SRP	4 puntos
Identifica correctamente las responsabilidades mezcladas	4 puntos
Muestra correctamente el ejemplo que sí cumple SRP	4 puntos

Analiza ventajas de aplicar SRP	4 puntos
Presenta el contenido de forma clara y ordenada	4 puntos
TOTAL SOBER 20 PTOS	